
자동화 도구 비교 분석 보고서



일 시	2026년 7월 29일
-----	--------------

성 명	박현준
-----	-----

소 속	코디세이 AI 네이티브
-----	--------------

< 목 차 >

1. 프로젝트 개요	3
1-1. 프로젝트 목적	3
1-2. 사용한 자동화 도구	3
1-3. 자동화 과제 선정 이유	4
1-4. 프로젝트 목표	5
2. 구현한 자동화 워크플로우	6
2-1. 자동화 과제 소개	6
2-2. 자동화 과제 선정 이유	6
2-3. 전체 워크플로우	7
2-4. Trigger, Action 및 조건 분기 구성	9
2-5. 요구사항 충족 여부	11
3. Make 와 Zapier 비교 분석	13
3-1. 구현 과정	13
3-2. 기능	14
3-3. 장단점	15
3-4. 프로젝트 수행 중 문제점	16
3-5. 종합 분석	18
4. 종합 평가 및 활용 의견	20
4-1. 프로젝트 결과	20
4-2. 활용 가능성	20
4-3. 최종 의견	21

1. 프로젝트 개요

1.1 프로젝트 목적

최근 다양한 분야에서 반복적인 업무를 자동화하여 업무 효율성을 향상시키기 위한 노코드(No-Code) 자동화 플랫폼의 활용이 증가하고 있습니다. 이러한 자동화 플랫폼은 별도의 프로그래밍 지식 없이도 다양한 서비스를 연동하여 반복 업무를 자동으로 수행할 수 있도록 지원하고 있으며, 개인뿐만 아니라 기업에서도 폭넓게 활용되고 있습니다.

본 프로젝트에서는 대표적인 노코드 자동화 플랫폼인 **Make**와 **Zapier**를 활용하여 동일한 자동화 워크플로우를 각각 구현하였습니다. 이후 두 플랫폼의 구현 방식과 사용 편의성, 조건 분기 방식, 무료 플랜 정책, 실행 결과 등을 비교·분석함으로써 각 플랫폼의 특징과 활용 가능성을 확인하고자 하였습니다.

또한 동일한 자동화 기능을 서로 다른 플랫폼에서 직접 구현하고 비교함으로써 각 플랫폼의 장단점을 객관적으로 분석하고, 실제 업무 환경에서 어떠한 상황에 적합한지를 평가하는 것을 본 프로젝트의 주요 목적으로 설정하였습니다.

1.2 사용한 자동화 도구

본 프로젝트에서는 다음의 두 가지 노코드 자동화 플랫폼을 활용하여 동일한 자동화 워크플로우를 구현하였습니다.

자동화 도구	설명	사용 목적
Make	노드(Node) 기반의 시각적 워크플로우를 제공하는 노코드 자동화 플랫폼	Google Calendar 일정 조회, 조건 분기, Slack 및 Discord 알림 전송
Zapier	단계별(Action) 방식의 노코드 자동화 플랫폼	동일한 자동화 워크플로우 구현 및 Make와의 비교 분석

자동화 결과를 확인하기 위하여 Google Calendar, Slack, Discord를 함께 연동하여 실제 알림이 정상적으로 전송되는지를 확인하였습니다.

1.3 자동화 과제 선정 이유

본 프로젝트에서는 Google Calendar에 등록된 일정 중 시작 시간이 임박한 일정에 대해 Slack과 Discord로 자동 알림을 전송하는 일정 관리 자동화 시스템을 구현 과제로 선정하였습니다.

일정 관리 업무는 학생과 직장인 모두에게 반복적으로 발생하는 업무 중 하나이며, 일정 시작 전에 직접 캘린더를 확인하는 과정을 자동화할 수 있다면 일정 관리의 효율성을 크게 향상시킬 수 있습니다.

또한 본 과제는 자동화 시스템에서 가장 기본적으로 사용되는 **Trigger**, **Action**, 조건 분기 (Filter/Router)를 모두 포함하고 있어 노코드 자동화 플랫폼의 핵심 기능을 비교하기에 적합하다고 판단하였습니다.

특히 동일한 업무를 Make와 Zapier에서 각각 구현함으로써 다음과 같은 항목들을 직접 비교할 수 있었습니다.

- 자동화 워크플로우 구성 방식
- UI 및 UX
- 조건 분기 구현 방식
- 실행 로그 확인 방법
- 무료 플랜 정책
- 실제 구현 난이도
- 확장성 및 유지보수 편의성

이를 통해 단순히 자동화를 구현하는 것뿐만 아니라 실제 사용자의 입장에서 두 플랫폼의 특징을 객관적으로 분석할 수 있었습니다.

1.4 프로젝트 목표

본 프로젝트에서는 다음과 같은 목표를 설정하여 자동화 시스템을 구현하였습니다.

- Google Calendar 일정 정보를 자동으로 조회하는 자동화 시스템을 구현한다.
- 일정 시작까지 남은 시간을 계산하여 조건 분기를 수행한다.
- 조건에 따라 Slack과 Discord로 자동 알림을 전송한다.
- 동일한 자동화 워크플로우를 Make와 Zapier에서 각각 구현한다.
- 두 플랫폼의 구현 방식과 기능을 비교·분석하여 각 플랫폼의 장단점을 도출한다.
- 실제 프로젝트 수행 과정에서 발생한 문제점을 분석하고 개선 방안을 제시한다.

2. 구현한 자동화 워크플로우

2.1 자동화 과제 소개

본 프로젝트에서는 Google Calendar에 등록된 일정 중 시작까지 60분 이내로 남은 일정을 자동으로 확인하고, 조건에 따라 Slack과 Discord로 일정 알림을 전송하는 자동화 워크플로우를 구현하였습니다.

자동화는 별도의 사용자 개입 없이 정해진 주기에 따라 실행되며, 일정 시작까지 남은 시간을 계산한 후 조건에 따라 서로 다른 경로를 통해 알림을 전송하도록 설계하였습니다.

동일한 자동화 구조를 Make와 Zapier에서 각각 구현하여 기능적인 차이와 사용성을 비교하였습니다.

2.2 자동화 과제 선정 이유

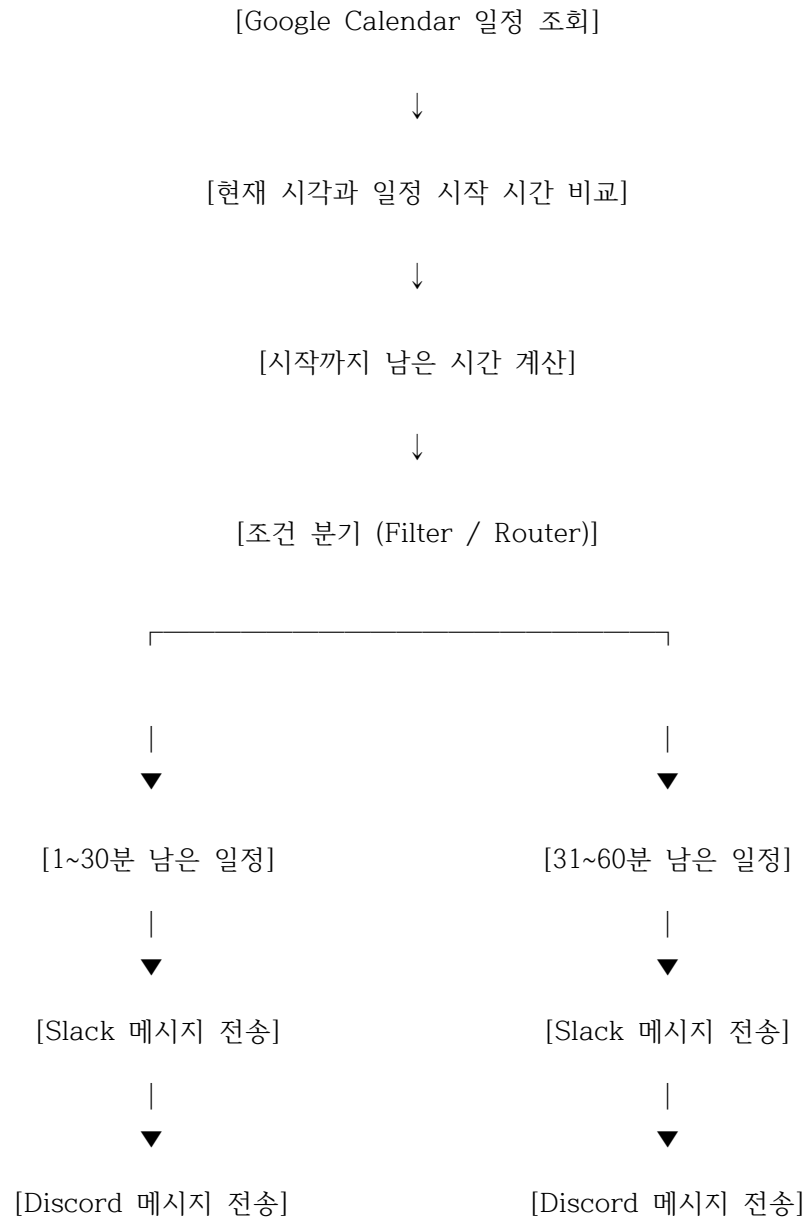
Google Calendar는 개인 일정 관리뿐만 아니라 팀 프로젝트와 업무 관리에서도 널리 사용되는 일정 관리 서비스입니다. 그러나 사용자가 직접 캘린더를 확인해야만 일정 시작 시간을 인지할 수 있다는 불편함이 존재합니다.

이를 해결하기 위해 일정이 임박한 경우 자동으로 메시지를 통해 알림을 전송하는 자동화 시스템을 구현하였습니다.

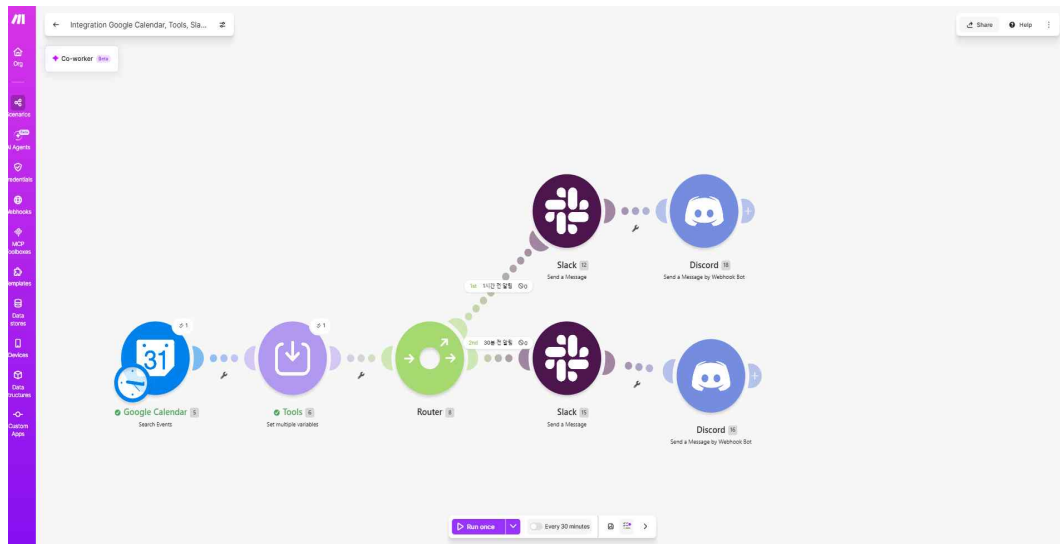
또한 일정 관리 자동화는 실제 활용도가 높은 업무일 뿐만 아니라 자동화 플랫폼의 핵심 기능인 Trigger, Action, 조건 분기를 모두 포함하고 있어 두 플랫폼을 비교하기에 적합한 사례라고 판단하였습니다.

2.3 전체 워크플로우

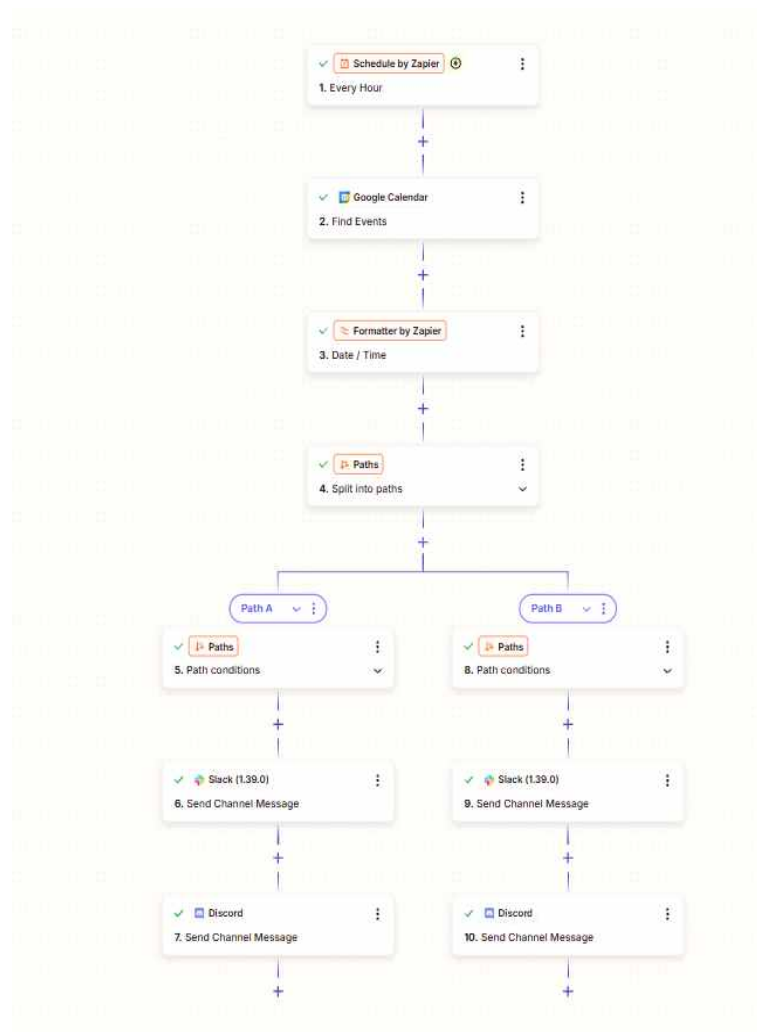
본 프로젝트에서 구현한 자동화 워크플로우는 다음과 같습니다.



Make 전체 워크플로우 화면



Zapier 전체 워크플로우 화면



2.4 Trigger, Action 및 조건 분기 구성

자동화 시스템은 Trigger, Action, 조건 분기 세 가지 핵심 요소를 중심으로 구성하였습니다.

(1) Trigger

Trigger는 자동화 워크플로우를 시작시키는 이벤트를 의미합니다.

본 프로젝트에서는 Google Calendar 일정 조회를 시작점으로 설정하였으며, Make와 Zapier 모두 일정한 주기에 따라 Google Calendar를 확인하도록 구성하였습니다.

- **Make** : Google Calendar 모듈을 이용하여 일정 조회
- **Zapier** : Schedule by Zapier를 이용하여 일정한 주기로 자동 실행 후 Google Calendar 조회

(2) Action

Action은 Trigger 이후 수행되는 실제 작업을 의미합니다.

본 프로젝트에서는 다음과 같은 작업을 수행하도록 구현하였습니다.

- 일정 시작까지 남은 시간 계산
- 조건 분기 수행
- Slack 메시지 자동 전송
- Discord 메시지 자동 전송

이를 통해 일정이 임박한 경우 사용자가 별도로 캘린더를 확인하지 않아도 자동으로 알림을 받을 수 있도록 구성하였습니다.

(3) 조건 분기(Filter / Router)

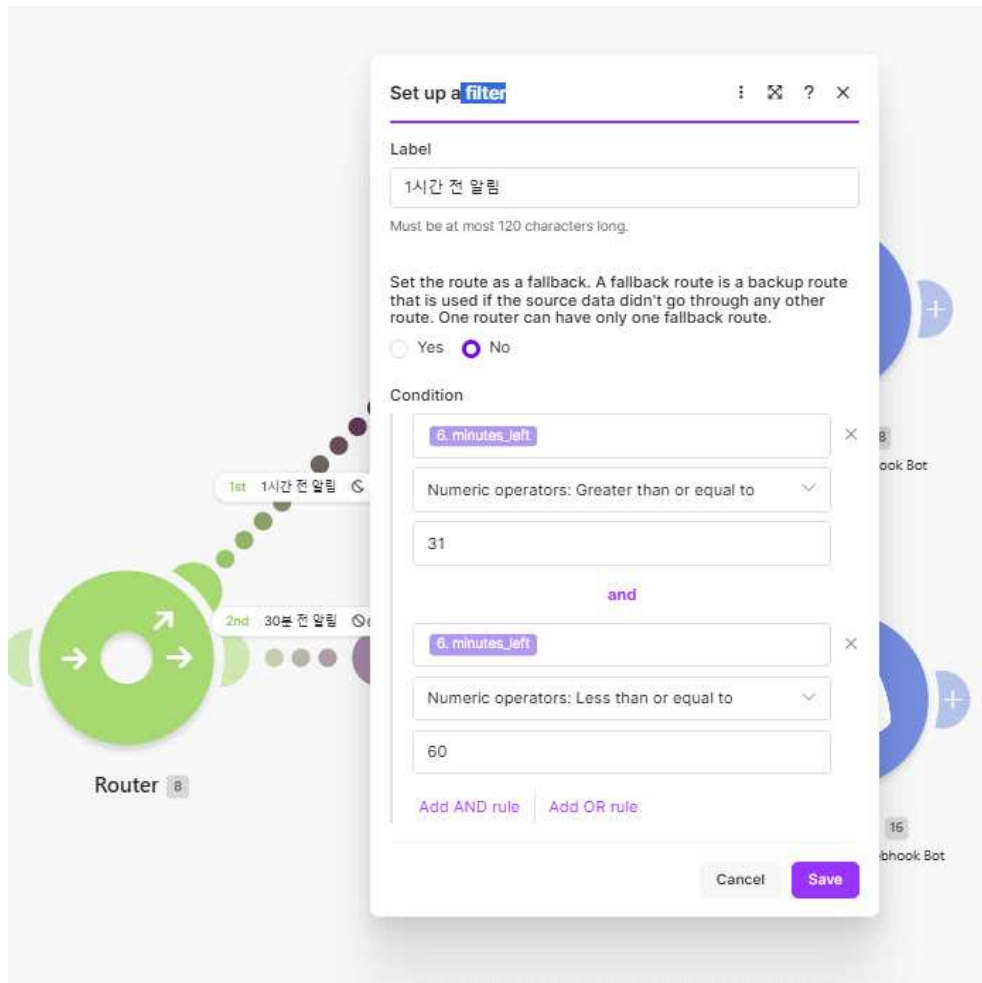
조건 분기는 일정 시작까지 남은 시간을 기준으로 서로 다른 경로를 수행하도록 설계하였습니다.

본 프로젝트에서는 다음과 같이 조건을 구성하였습니다.

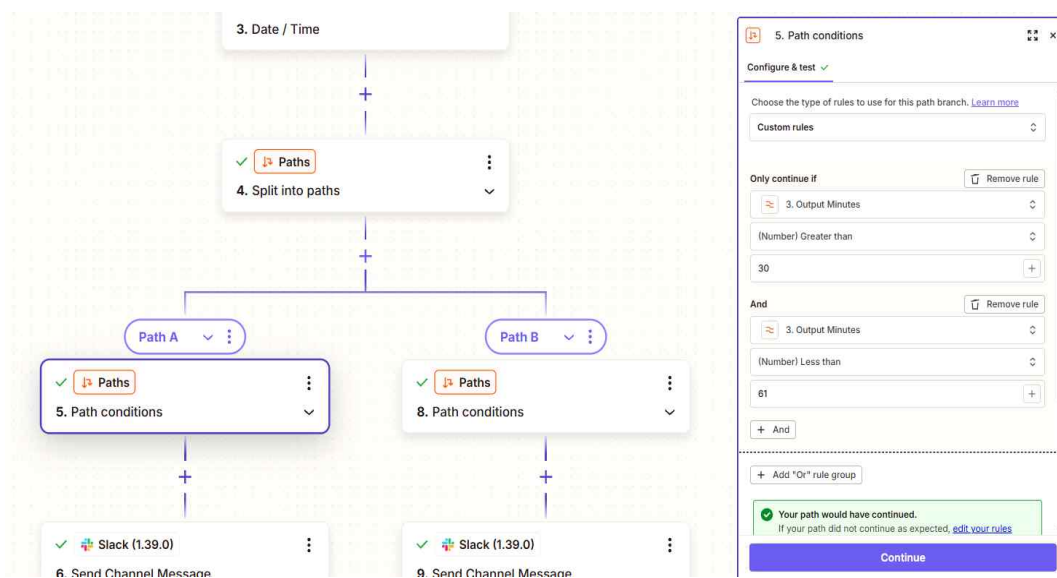
분기 조건	동작
31~60분 (일정 알림)	Slack 알림 → Discord 알림
1~30분 (일정 임박)	Slack 알림 → Discord 알림

Make에서는 **Router** 기능을 이용하여 조건 분기를 구현하였으며, Zapier에서는 **Paths** 기능을 이용하여 동일한 조건 분기를 구현하였습니다.

▼ Make 조건분기



▼ Zapier 조건분기



2.5 요구사항 충족 여부

본 프로젝트가 과제에서 제시한 요구사항을 충족하는지 확인한 결과는 다음과 같습니다.

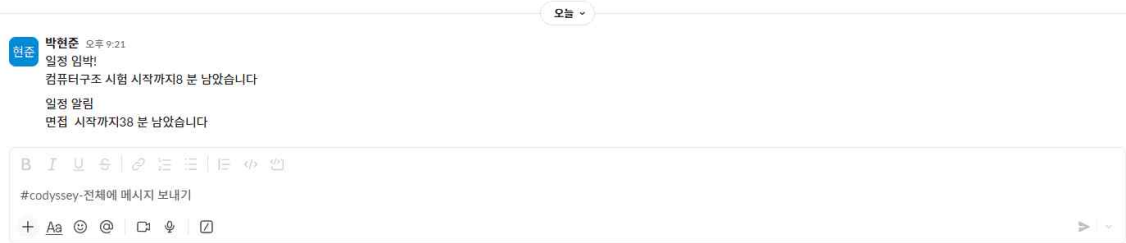
(1) Make 구현

요구사항	충족 여부	구현 내용
실제로 동작하는 자동화 워크플로우 구현	○	Google Calendar 일정 조회 후 Slack 및 Discord로 자동 알림이 정상적으로 전송되는 것을 확인하였습니다.
Trigger 1개 이상 포함	○	Google Calendar 모듈을 이용하여 일정 조회를 수행하였습니다.
Action 2개 이상 포함	○	Slack 메시지 전송과 Discord 메시지 전송을 구현하였습니다.
조건 분기(Filter/Router)	○	Router를 이용하여 1~30분과 31~60분 조건 분기를 구현하였습니다.
양쪽 분기 실행 결과 확인	○	각 조건에 대해 실제 Slack과 Discord 알림이 정상적으로 전송되는 것을 확인하였습니다.

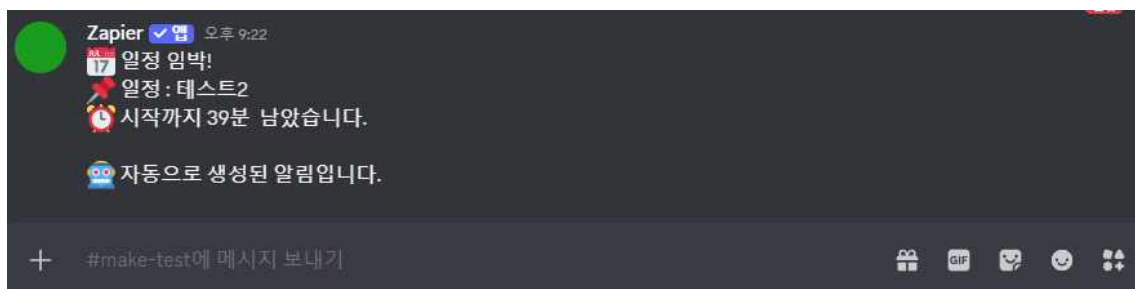
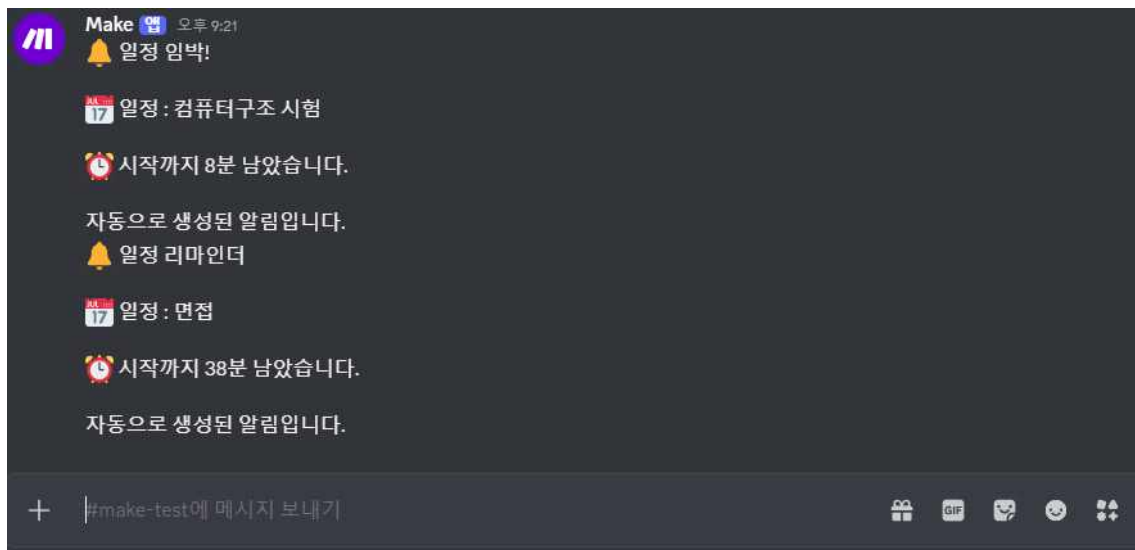
(2) Zapier 구현

요구사항	충족 여부	구현 내용
실제로 동작하는 자동화 워크플로우 구현	x	워크플로우 구성과 단계별 테스트는 완료하였으나 무료 플랜 Task 한도에 도달하여 최종 실행까지는 확인하지 못하였습니다.
Trigger 1개 이상 포함	○	Schedule by Zapier를 Trigger로 사용하였습니다.
Action 2개 이상 포함	○	Slack과 Discord 메시지 전송을 구현하였습니다.
조건 분기(Filter/Router)	○	Paths 기능을 이용하여 조건 분기를 구현하였습니다.
양쪽 분기 실행 결과 확인	○	Formatter의 Output Minutes 값을 이용하여 각 Path 조건이 정상적으로 동작하는 것을 확인하였습니다.

▼ Slack 알림 수신 화면 삽입



▼ Discord 알림 수신 화면 삽입



3. Make와 Zapier 비교 분석

3.1 구현 과정

본 프로젝트에서는 동일한 자동화 워크플로우를 Make와 Zapier에서 각각 구현하여 두 플랫폼의 구성 방식과 사용성을 비교하였습니다.

두 플랫폼 모두 Google Calendar와 연동하여 일정 정보를 조회하고, 일정 시작까지 남은 시간을 계산한 후 조건에 따라 Slack과 Discord로 알림을 전송하는 동일한 기능을 구현하였습니다. 그러나 워크플로우를 구성하는 방식과 자동화 흐름을 설계하는 과정에서는 차이가 존재하였습니다.

Make는 각 기능을 노드(Node) 형태로 연결하는 시각적인 방식으로 워크플로우를 구성하였습니다. 전체 자동화 흐름을 하나의 화면에서 확인할 수 있었으며, Router를 이용한 조건 분기 또한 직관적으로 표현되어 자동화 과정을 쉽게 이해할 수 있었습니다.

반면 **Zapier**는 단계별(Action) 방식으로 워크플로우를 구성하였습니다. 초기 설정은 비교적 간단하였으나, 시간 계산을 위해 Formatter를 추가적으로 사용해야 했으며, 조건 분기를 위해 Paths를 별도로 구성해야 하는 등 기능별 설정이 여러 단계로 나누어져 있었습니다. 동일한 기능을 구현하는 과정에서 Make는 전체 자동화 구조를 설계하는 데 유리하였고, Zapier는 간단한 자동화 구성을 빠르게 수행하는 데 적합하다는 점을 확인할 수 있었습니다.

3.2 기능

표 3-1. Make와 Zapier 기능 비교

비교 항목	Make	Zapier
UI / UX	노드(Node) 기반의 시각적 인터페이스를 제공하여 전체 워크플로우를 한 화면에서 확인할 수 있었습니다.	단계별(Action) 리스트 형태로 구성되어 초보자도 쉽게 접근할 수 있었습니다.
설정 난이도	초기에는 다양한 기능으로 인해 학습이 필요하였으나, 익숙해진 이후에는 복잡한 자동화도 효율적으로 구성할 수 있었습니다.	직관적인 화면 구성으로 간단한 자동화는 빠르게 구현할 수 있었으나 복잡한 워크플로우에서는 단계가 많아지는 특징이 있었습니다.
조건 분기	Router를 이용하여 여러 개의 조건을 자유롭게 구성할 수 있었습니다.	Paths 기능을 이용하여 조건 분기를 구현할 수 있었으나 Router보다 표현력이 제한적이었습니다.
시간 계산	Tools 모듈을 이용하여 다양한 계산식을 자유롭게 구성할 수 있었습니다.	Formatter를 별도로 추가하여 시간을 계산해야 하였습니다.
연동 서비스	Google, Slack, Discord 등 다양한 서비스와 세부적인 설정이 가능하였습니다.	대표적인 클라우드 서비스를 중심으로 폭넓은 연동을 지원하였습니다.
실행 로그	History 기능을 통해 각 모듈의 입력값과 출력값을 단계별로 확인할 수 있었습니다.	Zap History를 통해 실행 결과를 확인할 수 있었으나 무료 플랜에서는 일부 기능이 제한되었습니다.
무료 플랜	무료 사용량이 비교적 넉넉하여 반복 테스트를 수행하기에 적합하였습니다.	Task 사용량 제한으로 인해 반복 테스트 수행에 제약이 있었습니다.
확장성	복잡한 자동화와 다단계 워크플로우 구성에 적합하였습니다.	간단한 자동화에는 적합하였으나 복잡한 자동화에서는 단계가 많아지는 경향이 있었습니다.

3.3 장단점

표 3-2. Make와 Zapier 장단점 비교

구분	Make	Zapier
장점	시각적인 워크플로우 구성이 가능하였습니다.	사용법이 직관적이었습니다.
	복잡한 조건 분기를 쉽게 구현할 수 있었습니다.	초보자도 쉽게 자동화를 구축할 수 있었습니다.
	무료 플랜 사용량이 비교적 넉넉하였습니다.	Google 서비스와의 연동이 간편하였습니다.
	실행 로그를 상세하게 확인할 수 있었습니다.	설정 과정이 단순하여 빠르게 자동화를 구성할 수 있었습니다.
단점	초기 학습 시간이 다소 필요하였습니다.	무료 플랜 Task 제한이 비교적 적었습니다.
	설정 항목이 많아 처음에는 다소 복잡하게 느껴졌습니다.	Formatter 등 추가 모듈이 필요한 경우가 많았습니다.
	기능이 많아 초보자가 접근하기에는 어려울 수 있었습니다.	복잡한 조건 분기에서는 단계 수가 증가하는 특징이 있었습니다.

3.4 프로젝트 수행 중 문제점

프로젝트를 수행하는 과정에서 가장 큰 문제점은 **Zapier 무료 플랜의 Task 사용량 제한**이었습니다.

본 프로젝트에서는 Google Calendar 일정 조회, 시간 계산(Formatter), 조건 분기(Paths), Slack 메시지 전송, Discord 메시지 전송으로 이어지는 자동화 워크플로우를 구성하였습니다. 그러나 테스트를 반복하는 과정에서 무료 플랜에서 제공하는 월간 Task를 모두 소진하게 되었고, 이후 Formatter 단계부터 **"You've met your plan limit"** 오류가 발생하여 워크플로우가 더 이상 정상적으로 실행되지 않았습니다.

이로 인해 Google Calendar 일정 조회와 Formatter, Paths 단계까지의 동작은 확인할 수 있었지만, Slack 및 Discord로 이어지는 최종 Action은 무료 플랜 제한으로 인해 실제 실행 결과를 검증할 수 없었습니다. 즉, 워크플로우의 설계와 구성은 완료되었으나, 무료 플랜의 사용량 제한으로 인해 실제 운영 환경과 동일한 조건에서 충분한 테스트를 수행하지 못하였습니다.

이러한 경험을 통해 노코드 자동화 플랫폼을 선택할 때에는 기능의 다양성뿐만 아니라 **무료 플랜의 사용량 정책과 과금 구조 또한 중요한 평가 요소**임을 확인할 수 있었습니다. 특히 반복적인 테스트가 필요한 개발 및 검증 단계에서는 무료 플랜의 Task 제한이 프로젝트 진행에 직접적인 영향을 줄 수 있으며, 자동화 규모가 커질수록 이러한 제약은 더욱 크게 나타날 수 있습니다.

반면 Make는 동일한 프로젝트를 수행하는 과정에서 무료 플랜 범위 내에서도 반복적인 테스트와 디버깅을 비교적 자유롭게 수행할 수 있었으며, 실행 이력과 시각적인 워크플로우를 활용하여 문제를 확인하고 수정하는 과정도 원활하게 진행할 수 있었습니다. 따라서 본 프로젝트를 통해 두 플랫폼 모두 동일한 자동화 기능을 구현할 수 있음을 확인하였으나, 무료 플랜 환경에서 반복적인 개발과 테스트를 수행하는 경우에는 **Make가 Zapier보다 프로젝트 수행에 더욱 적합한 환경을 제공하였다고** 판단하였습니다.

▼ Zapier 무료 플랜 제한

Search Ctrl ⬆ ⬆ ⬆

Zap runs

A log of all automation activity within your Zap, which can help with troubleshooting. [Learn more](#). Access these runs in [Zap History](#).

Search All statuses Last 30 days

5 runs based on applied filters

Select runs to replay steps

☐ Runs to replay

- ☐ On hold Test 7월 28, 2026 09:24 오후
- ☐ On hold Test 7월 28, 2026 05:49 오후
- ☐ On hold Test 7월 28, 2026 05:44 오후
- ☐ On hold Test 7월 23, 2026 04:36 오후
- ☐ On hold Test 7월 23, 2026 04:28 오후

Exit run view

< 1 >

Version: 1 Run ID: 01aafbe-1457-a29f-a61e-44153506f694 On hold: 7월 28, 2026 09:24:16 오후

✓ **Schedule by Zapier**

1. Every Hour

Found 1 new Hour in Schedule by Zapier.

✓ **Google Calendar**

2. Find Events

Found Event in Google Calendar.

II **Formatter by Zapier**

3. Date / Time

This Formatter by Zapier step is pending because you've met your plan limit.

Paths

4. Split into paths

Path A

Path B

Paths

5. Path conditions

Did not attempt to run this path.

Paths

8. Path conditions

Did not attempt to run this path.

3.5 종합 분석

동일한 자동화 기능을 두 플랫폼에서 구현한 결과, 기능적인 측면에서는 두 플랫폼 모두 충분히 동일한 자동화를 구현할 수 있었습니다.

그러나 자동화를 설계하고 테스트하는 과정에서는 차이가 존재하였습니다. Make는 Router를 이용한 조건 분기와 시각적인 워크플로우 구성이 매우 직관적이었으며, 실행 로그를 상세하게 확인할 수 있어 오류를 빠르게 수정할 수 있었습니다.

반면 Zapier는 기본적인 자동화 구성은 매우 간단하였으나, 시간 계산을 위해 Formatter를 추가해야 했고 무료 플랜의 Task 제한으로 인해 반복 테스트가 어려웠습니다.

특히 본 프로젝트와 같이 여러 번의 테스트가 필요한 자동화 프로젝트에서는 무료 사용량이 넉넉한 Make가 보다 효율적인 플랫폼이라고 판단하였습니다.

반대로 간단한 자동화나 비교적 작은 규모의 프로젝트에서는 Zapier 역시 충분히 활용 가능한 플랫폼이라고 판단하였습니다.

4. 종합 평가 및 활용 의견

4.1 프로젝트 결과

본 프로젝트에서는 Make와 Zapier를 이용하여 동일한 자동화 워크플로우를 각각 구현하고 두 플랫폼의 기능과 사용성을 비교·분석하였습니다.

Make에서는 Google Calendar 일정 조회, 조건 분기, Slack 및 Discord 알림 전송까지 전체 자동화가 정상적으로 수행되는 것을 확인하였습니다.

Zapier에서도 동일한 자동화 구조를 구현할 수 있었으며 Trigger, Formatter, Paths를 이용한 조건 분기까지 정상적으로 구성하였습니다. 다만 무료 플랜의 Task 제한으로 인해 최종 알림 전송 단계까지는 실행하지 못하였습니다.

그럼에도 불구하고 워크플로우 구성 방식과 조건 분기 방법, 자동화 설계 방식은 충분히 비교·분석할 수 있었으며 두 플랫폼의 특징을 이해하는 데에는 큰 문제가 없었습니다.

4.2 활용 가능성

이번 프로젝트에서 구현한 자동화 시스템은 일정 관리뿐만 아니라 다양한 업무 환경에서도 활용할 수 있을 것으로 판단됩니다.

예를 들어 프로젝트 일정 관리, 팀 회의 알림, 마감 일정 관리, 과제 제출 일정 관리 등 다양한 업무에 적용할 수 있으며, Slack이나 Discord뿐만 아니라 Gmail, Telegram, Microsoft Teams 등 다양한 서비스와 연계하여 활용할 수 있습니다.

또한 조건 분기를 추가하거나 AI 서비스를 연동한다면 일정 요약, 우선순위 분류, 자동 보고서 생성 등 더욱 다양한 자동화 시스템으로 확장할 수 있을 것으로 기대됩니다.

4.3 최종 의견

본 프로젝트를 수행하면서 동일한 자동화 기능이라도 플랫폼에 따라 워크플로우 구성 방식과 사용 경험에 차이가 있다는 점을 확인할 수 있었습니다.

Make는 시각적인 인터페이스와 자유로운 조건 분기, 넉넉한 무료 플랜을 제공하여 복잡한 자동화 프로젝트를 구현하는 데 적합하였습니다.

반면 Zapier는 직관적인 사용자 인터페이스와 간편한 설정 방식을 제공하여 초보자도 쉽게 자동화를 구축할 수 있었으나, 무료 플랜의 Task 제한으로 인해 반복 테스트와 장시간의 프로젝트 수행에는 다소 제약이 있었습니다.

따라서 복잡한 자동화와 다양한 조건 분기가 필요한 프로젝트에는 **Make**를 사용하는 것이 적합하며, 비교적 간단한 반복 업무 자동화나 빠른 프로토타입 구현에는 **Zapier**가 적합하다고 판단하였습니다.

이번 프로젝트를 통해 노코드 자동화 플랫폼의 기본 원리를 이해할 수 있었으며, 자동화 시스템 설계 능력뿐만 아니라 문제 해결 과정과 플랫폼 선택 기준에 대해서도 학습할 수 있었습니다. 앞으로는 생성형 AI와 노코드 자동화를 연계하여 보다 지능적인 업무 자동화 시스템을 구축하는 방향으로 확장할 수 있을 것으로 기대됩니다.